LAB ASSIGNMENT-7

1. Class polygon contains data member width and height and public method set_value() to assign values to width and height. Class Rectangle and Triangle are inherited from polygon class. Both the classes contain public method calculate_area() to calculate the area of Rectangle and Triangle. Use base class pointer to access the derived class object and show the area calculated.

CODE:

```
#include <iostream>
using namespace std;

class Polygon {
protected:
   int width, height;

public:
   void set_value(int w, int h) {
      width = w;
      height = h;
   }

   virtual int calculate_area() {
      return 0;
   }
};

class Rectangle : public Polygon {
public:
   int calculate_area() {
      return width * height;
   }
};

class Triangle : public Polygon {
public:
   int calculate_area() {
      return 0.5 * width * height;
   }
};

int main() {
   Polygon *p;

   Rectangle rect;
   Triangle tri;

   p = &rect;
```

```
    p->set_value(10, 5);

    cout << "Area of Rectangle: "
         << p->calculate_area() << endl;

    p = &tri;
    p->set_value(10, 5);

    cout << "Area of Triangle: "
         << p->calculate_area() << endl;

    return 0;
}
```

2. Write a program to create a class shape with functions area and display to find area and display the name of the shape and other essential component of the class. Create derived classes circle, rectangle and triangle each having overridden functions area and display. Write a program to find and display the area of circle, rectangle and triangle.

CODE:

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void area() {
        cout << "Area not defined" << endl;
    }

    virtual void display() {
        cout << "This is a shape" << endl;
    }
};

class Circle : public Shape {
    float radius;

public:
    Circle(int r) {
        radius = r;
    }

    void area() {
        cout << "Area of Circle: "
             << 3.14 * radius * radius << endl;
    }
```

```
    void display() {
        cout << "Shape: Circle" << endl;
    }
};

class Rectangle : public Shape {
    float length, breadth;

public:
    Rectangle(float l, float b) {
        length = l;
        breadth = b;
    }

    void area() {
        cout << "Area of Rectangle: "
             << length * breadth << endl;
    }

    void display() {
        cout << "Shape: Rectangle" << endl;
    }
};

class Triangle : public Shape {
    float base, height;

public:
    Triangle(float b, float h) {
        base = b;
        height = h;
    }

    void area() {
        cout << "Area of Triangle: "
             << 0.5 * base * height << endl;
    }

    void display() {
        cout << "Shape: Triangle" << endl;
    }
};

int main() {
    Shape *s;

    Circle c(5);
    Rectangle r(10, 4);
    Triangle t(6, 3);
```

```
   s = &c;
   s->display();
   s->area();

   s = &r;
   s->display();
   s->area();

   s = &t;
   s->display();
   s->area();

   return 0;
}
```

3. Write a C++ program to compute area of right-angle triangle, equilateral triangle, isosceles triangle using function overloading concept.

CODE:

```
#include <iostream>
#include <cmath>
using namespace std;

// Right-angled triangle
double area(double base, double height) {
   return 0.5 * base * height;
}

// Equilateral triangle
double area(double side) {
   return (sqrt(3) / 4) * side * side;
}

// Isosceles triangle
double area(double equalSide, double base) {
   double height =
      sqrt(equalSide * equalSide - (base * base) / 4);

   return 0.5 * base * height;
}

int main() {
   double b = 6, h = 4;
   double side = 5;
   double equalSide = 5, base = 6;

   cout << "Right-angled Triangle Area: "
```

```
        << area(b, h) << endl;

    cout << "Equilateral Triangle Area: "
        << area(side) << endl;

    cout << "Isosceles Triangle Area: "
        << area(equalSide, base) << endl;

    return 0;
}
```

4. Write a program with Student as abstract class and create derive classes Engineering, Medicine and Science from base class Student. Create the objects of the derived classes and process them and access them using array of pointer of type base class Student.

CODE:

```
// 4. Abstract Class Student

#include <iostream>
using namespace std;

class Student {
protected:
    string name;

public:
    Student(string n) {
        name = n;
    }

    virtual void display() = 0;
};

class Engineering : public Student {
public:
    Engineering(string n) : Student(n) {
    }

    void display() {
        cout << "Engineering Student: "
            << name << endl;
    }
};

class Medicine : public Student {
public:
    Medicine(string n) : Student(n) {
    }

    void display() {
        cout << "Medicine Student: "
            << name << endl;
    }
};

class Science : public Student {
public:
```

```
    Science(string n) : Student(n) {
    }

    void display() {
        cout << "Science Student: "
             << name << endl;
    }
};

int main() {
    Student *students[3];

    students[0] = new Engineering("Alice");
    students[1] = new Medicine("Bob");
    students[2] = new Science("Charlie");

    for (int i = 0; i < 3; i++) {
        students[i]->display();
    }

    for (int i = 0; i < 3; i++) {
        delete students[i];
    }

    return 0;
}
```

5. Create a class Time with three private variables int h,m,s; Create a function to overload '+' operator to add two time variables.

```
int main(){
Time t1(5,15,34),t2(9,53,58),t3;
t3 = t1 + t2;
t3.show();
}
```

CODE:

```
// 5. Operator Overloading (+) for Time

#include <iostream>
using namespace std;

class Time {
    int h, m, s;

public:
    Time(int hh = 0, int mm = 0, int ss = 0) {
        h = hh;
        m = mm;
        s = ss;
    }

    Time operator +(Time t) {
        Time temp;
```

```
        temp.s = s + t.s;
        temp.m = m + t.m;
        temp.h = h + t.h;

        if (temp.s >= 60) {
            temp.s -= 60;
            temp.m++;
        }

        if (temp.m >= 60) {
            temp.m -= 60;
            temp.h++;
        }

        return temp;
    }

    void show() {
        cout << "Time: "
             << h << " : "
             << m << " : "
             << s << endl;
    }
};

int main() {
    Time t1(5, 15, 34),
         t2(9, 53, 58),
         t3;

    t3 = t1 + t2;

    t3.show();

    return 0;
}
```

6. Define a class STRING and overload == to compare two strings and + operator for concatenation of two strings.

CODE:

```
// 6. STRING Class Operator Overloading

#include <iostream>
using namespace std;

class STRING {
private:
    string str;

public:
```

```
    STRING(string s = "") {
        str = s;
    }

    bool operator ==(STRING s) {
        return (str == s.str);
    }

    STRING operator +(STRING s) {
        return STRING(str + s.str);
    }

    void display() {
        cout << str << endl;
    }
};

int main() {
    STRING s1("Hello "),
           s2("World"),
           s3;

    s3 = s1 + s2;

    cout << "Concatenated String: ";
    s3.display();

    if (s1 == s2) {
        cout << "Strings are Equal" << endl;
    }
    else {
        cout << "Strings are Not Equal" << endl;
    }

    return 0;
}
```

7. Write a program in C++ to create a class matrix and overload * operator using friend function to multiply two matrices.

CODE:

```
// 7. Matrix Multiplication using Friend Function

#include <iostream>
using namespace std;

class Matrix {
private:
    int a[10][10], row, col;

public:
    void getData(int r, int c) {
```

```
    row = r;
    col = c;

    cout << "Enter elements:\n";

    for (int i = 0; i < row; i++) {
      for (int j = 0; j < col; j++) {
        cin >> a[i][j];
      }
    }
  }

  void display() {
    for (int i = 0; i < row; i++) {
      for (int j = 0; j < col; j++) {
        cout << a[i][j] << " ";
      }

      cout << endl;
    }
  }

  friend Matrix operator *(Matrix m1, Matrix m2);
};

Matrix operator *(Matrix m1, Matrix m2) {
  Matrix temp;

  if (m1.col != m2.row) {
    cout << "Matrix multiplication not possible!"
         << endl;

    temp.row = temp.col = 0;
    return temp;
  }

  temp.row = m1.row;
  temp.col = m2.col;

  for (int i = 0; i < temp.row; i++) {
    for (int j = 0; j < temp.col; j++) {

      temp.a[i][j] = 0;

      for (int k = 0; k < m1.col; k++) {
        temp.a[i][j] +=
          m1.a[i][k] * m2.a[k][j];
      }
    }
  }
```

```
    return temp;
}

int main() {
    Matrix m1, m2, result;

    int r1, c1, r2, c2;

    cout << "Enter rows and columns of first matrix: ";
    cin >> r1 >> c1;

    m1.getData(r1, c1);

    cout << "Enter rows and columns of second matrix: ";
    cin >> r2 >> c2;

    m2.getData(r2, c2);

    result = m1 * m2;

    cout << "Resultant Matrix:\n";
    result.display();

    return 0;
}
```

8. Overload '[]' to check array index out of bounds problem at run time.

CODE:

```
#include <iostream>
using namespace std;

class SafeArray {
private:
    int arr[10];
    int size;

public:
    SafeArray(int s = 10) {
        size = s;
    }

    int& operator [](int index) {

        if (index < 0 || index >= size) {
            cout << "Error: Index out of bounds!"
                 << endl;

            exit(1);
```

```
        }

        return arr[index];
    }
};

int main() {
    SafeArray a(5);

    for (int i = 0; i < 5; i++) {
        a[i] = i * 10;
    }

    cout << "Array elements:\n";

    for (int i = 0; i < 5; i++) {
        cout << a[i] << " ";
    }

    cout << endl;

    cout << "Accessing invalid index:\n";
    cout << a[7];

    return 0;
}
```

9. Overload '()' to input arbitrary number of input arguments for an object.

CODE:

```
#include <iostream>
#include <cstdarg>
using namespace std;

class Numbers {
private:
    int sum;

public:
    Numbers() {
        sum = 0;
    }

    void operator()(int count, ...) {

        va_list args;

        va_start(args, count);
```

```
        sum = 0;

        for (int i = 0; i < count; i++) {
            sum += va_arg(args, int);
        }

        va_end(args);
    }

    void display() {
        cout << "Sum = " << sum << endl;
    }
};

int main() {
    Numbers obj;

    obj(3, 10, 20, 30);
    obj.display();

    obj(5, 1, 2, 3, 4, 5);
    obj.display();

    return 0;
}
```

10. Write a program in C++ to overload input operator (>>) and output operator (<<).

CODE:

```
#include <iostream>
#include <cstdarg>
using namespace std;

class Numbers {
private:
    int sum;

public:
    Numbers() {
        sum = 0;
    }

    void operator()(int count, ...) {

        va_list args;

        va_start(args, count);
```

```
        sum = 0;

        for (int i = 0; i < count; i++) {
            sum += va_arg(args, int);
        }

        va_end(args);
    }

    void display() {
        cout << "Sum = " << sum << endl;
    }
};

int main() {
    Numbers obj;

    obj(3, 10, 20, 30);
    obj.display();

    obj(5, 1, 2, 3, 4, 5);
    obj.display();

    return 0;
}
```

11. Write a program to convert basic data type (float) to user defined data type (object). class Test {

```
private
     :
   //…
 publi
    c:
  Test(data_type) {
     // conversion code
    }
};
```

CODE:

```
#include <iostream>
using namespace std;

class Test {
private:
    float value;

public:
    Test(float x) {
        value = x;
    }
```

```
    void display() {
        cout << "Value = "
             << value << endl;
    }
};

int main() {
    float num = 5.75;

    Test obj = num;

    obj.display();

    return 0;
}
```

12 . Write a program to convert UDT to basic data type (float) class Test {

```
public:
 operator data_type() {
    // conversion code
    }
};
```

CODE:

```
#include <iostream>
using namespace std;

class Test {
private:
    float value;

public:
    Test(float x) {
        value = x;
    }

    operator float() {
        return value;
    }
};

int main() {
    Test obj(7.25);

    float num;

    num = obj;

    cout << "Value = "
         << num << endl;
```

```
    return 0;
}
```

13. How will you convert one UDT to another UDT. For example conversion of polar to cartesian system.

Polar
p(10,5);
Cartesian c =
p; c.show();

CODE:

```
#include <iostream>
#include <cmath>
using namespace std;

class Polar {
private:
    float r, theta;

public:
    Polar(float rad, float angle) {
        r = rad;
        theta = angle;
    }

    float getR() {
        return r;
    }

    float getTheta() {
        return theta;
    }
};

class Cartesian {
private:
    float x, y;

public:
    Cartesian(Polar p) {
        x = p.getR() * cos(p.getTheta());
        y = p.getR() * sin(p.getTheta());
    }

    void show() {
        cout << "x = " << x
             << ", y = " << y << endl;
    }
};

int main() {
    Polar p(10, 5);

    Cartesian c = p;
```

```
    c.show();

    return 0;
}
```